

Project Design Phase-II Technology Stack (Architecture & Stack)

Date	2 October 2026
Team ID	SWTID-2026-6545
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Example: Order processing during pandemics for offline mode

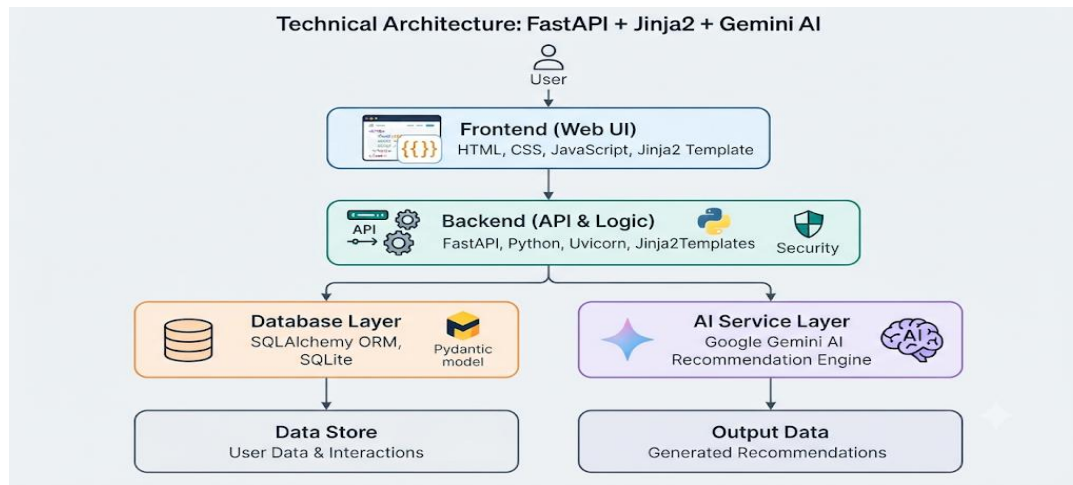


Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	Web interface for planner selection, budget/preferences input, recommendation display, and optional image upload.	HTML, CSS, Jinja2 Template, JavaScript
2.	Application Logic-1	Handles planner selection, input validation, budget processing, and recommendation request orchestration.	Python, FastAPI + SQLAlchemy
3.	Application Logic-2	Builds AI prompts and processes budget, category, preferences, and contextual inputs.	Gemini API
4.	Application Logic-3	Formats recommendations, filters results, and presents suitable product/service options.	Python, FastAPI
5.	Database	The core recommendation flow does not require a mandatory relational database; session data can be handled by the application.	SQLite - stores user accounts, planner inputs, recommendation history, and other persistent application data.
6.	Cloud Database	No cloud database is required for the core prototype recommendation workflow.	Not required for core prototype
7.	File Storage	Temporary handling of optional outfit/product images during the recommendation process.	Local/temporary file handling

8.	External API-1	AI service used to interpret user requirements and generate personalized recommendations.	Google Gemini API
9.	External API-2	External product/service platforms used as recommendation sources and destination links.	Supported shopping/service platform links
10.	Machine Learning Model	Generative AI model used for budget interpretation, preference understanding, and recommendation generation.	Gemini model via Gemini API
11.	Infrastructure (Server / Cloud)	Frontend and FastAPI backend can be developed locally and deployed as a web application.	Local development; web hosting + Python/FastAPI server

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Open-source frameworks are used for the web UI and backend API layer.	HTML,CSS,JS, FastAPI, Python
2.	Security Implementations	Use environment variables for API credentials, protected routes where required, input validation, and HTTPS in deployment.	Environment variables, authentication/session controls, HTTPS, input validation
3.	Scalable Architecture	Frontend, backend, AI service, and external platform layer are separated so components can be extended or replaced independently.	React + FastAPI + modular service architecture

4.	Performance	Keep the interface responsive, validate inputs before AI calls, use focused prompts, and avoid unnecessary recommendation requests.	Responsive UI; request validation; efficient API calls
5.	Availability	Frontend and backend can be deployed independently; the core recommendation flow does not depend on a local-only database.	Cloud/web deployment; external Gemini service